

Лабораторная работа №3: C++ & UNIX: CLI-калькулятор и рекурсия

Лана Глазко
z 3344, 3 курс

23 мая 2025 г.

Цель работы

Познакомиться с реализацией программ на C++ с использованием командной строки (CLI), а также с рекурсивным программированием. Получить опыт работы с арифметическими операциями, функциями, условиями, циклами и аргументами командной строки.

Задания

1. C++ CLI CALC: Создать программу CALC с интерфейсом CLI.
2. C++ RECURSION: Решить задачу ханойской башни с использованием рекурсии

Структура проекта

```
cpp_unix_project/  
  lab_03/  
    build/  
    cmake/  
    doc/  
    src/
```

1 [C++ CLI CALC] Создать программу CALC с интерфейсом CLI

```
1 #include <iostream>  
2 #include <string>  
3 #include <cstdlib>  
4  
5 using namespace std;  
6
```

```

7 double add(double a, double b) {
8     return a + b;
9 }
10
11 double subtract(double a, double b) {
12     return a - b;
13 }
14
15 double power(double base, double exponent) {
16     int intExponent = static_cast<int>(exponent);
17     double result = 1.0;
18     for (int i = 0; i < intExponent; ++i) {
19         result *= base;
20     }
21     return result;
22 }
23
24 int main(int argc, char* argv[]) {
25     if (argc != 4) {
26         cerr << "Usage: ./calc [+ , -, ^] operand1 operand2\n";
27         return 1;
28     }
29
30     string operation = argv[1];
31     double operand1 = atof(argv[2]);
32     double operand2 = atof(argv[3]);
33
34     if (operation == "+") {
35         cout << "Result: " << add(operand1, operand2) << "\n";
36     } else if (operation == "-") {
37         cout << "Result: " << subtract(operand1, operand2) << "\n";
38     } else if (operation == "^") {
39         int intExponent = static_cast<int>(operand2);
40         if (operand2 != intExponent || intExponent < 0) {
41             cerr << "Error: exponent must be a non-negative integer.\n";
42             return 1;
43         }
44         cout << "Result: " << power(operand1, operand2) << "\n";
45     } else {
46         cerr << "Unknown operation. Use +, -, or ^\n";
47         return 1;
48     }
49
50     return 0;
51 }

```

Листинг 1: Файл calc.cpp

Пояснение к коду

Калькулятор реализован через функции: `add`, `subtract`, `power`. Ввод осуществляется через аргументы командной строки. Проверяется корректность ввода и допустимость степени. Возведение в степень реализовано вручную через цикл.

Компиляция и запуск

```
1 cd /mnt/c/Users/lana/OneDrive/Документы/cpp_unix_project/lab_03
2 g++ src/calc.cpp -o build/calc
3
4 ./build/calc + 3 4
5 Result: 7
6
7 ./build/calc ^ 3 2
8 Result: 9
9
10 ./build/calc - 5 100
11 Result: -95
12
13 ./build/calc + 3
14 Usage: ./calc [+ , - , ^] operand1 operand2
15
16 ./build/calc 8 8 8
17 Unknown operation. Use +, -, or ^
18
19 ./build/calc ^ 2 -1
20 Error: exponent must be a non-negative integer.
```

Вывод

Создан CLI-калькулятор с базовыми арифметическими операциями. Программа компилируется и успешно тестируется через командную строку. Использование CLI делает её удобной для автоматизированного использования и практики C++.

2 [C++ RECURSION] Решить задачу Ханойской башни с использованием рекурсии

Условие

Решить задачу Ханойской башни для **произвольного количества колец** n , используя **рекурсивный алгоритм**. По правилам, за один ход можно переносить только одно кольцо и нельзя класть большее кольцо на меньшее. В задаче участвуют три стержня.

Результат (последовательность шагов) необходимо сохранять в **двусвязный список**. После завершения работы — вывести список на экран и корректно освободить всю выделенную память.

Программа должна поддерживать **CLI-интерфейс**: принимать количество колец и названия стержней как аргументы командной строки, либо — ввести их вручную при отсутствии аргументов.

Алгоритм решения

Алгоритм решения задачи Ханойской башни основан на классическом рекурсивном подходе:

- Если $n = 1$, просто переместить диск с начального стержня на целевой.
- Иначе:
 1. Переместить $n - 1$ дисков со стартового стержня на вспомогательный.
 2. Переместить самый большой диск на целевой стержень.
 3. Переместить $n - 1$ дисков с вспомогательного на целевой.

Последовательность шагов сохраняется в двусвязный список, каждая вершина которого содержит текст одного шага (например, “Move disk 1 from A to C”).

Программная реализация

Файл: src/hanoi.cpp

```
1 #include <iostream>
2 #include <string>
3 #include <cstdlib> // для atoi
4
5 using namespace std;
6
7 struct Node {
8     string move;
9     Node* prev;
10    Node* next;
11
12    Node(string m) {
13        move = m;
14        prev = NULL;
15        next = NULL;
16    }
17 };
18
19 Node* head = NULL;
20 Node* tail = NULL;
21
22 void addMove(string move) {
23     Node* newNode = new Node(move);
24     if (!head) {
25         head = tail = newNode;
26     } else {
27         tail->next = newNode;
```

```

28     newNode->prev = tail;
29     tail = newNode;
30 }
31 }
32
33 void solveHanoi(int n, char from, char to, char aux) {
34     if (n == 1) {
35         addMove("Move disk 1 from " + string(1, from) + " to " + string(1, to));
36         return;
37     }
38     solveHanoi(n - 1, from, aux, to);
39     addMove("Move disk " + to_string(n) + " from " + string(1, from) + " to " +
40         string(1, to));
41     solveHanoi(n - 1, aux, to, from);
42 }
43
44 void printMoves() {
45     Node* current = head;
46     int i = 1;
47     while (current) {
48         cout << i++ << ". " << current->move << endl;
49         current = current->next;
50     }
51 }
52
53 void clearList() {
54     Node* current = head;
55     while (current) {
56         Node* next = current->next;
57         delete current;
58         current = next;
59     }
60     head = tail = NULL;
61 }
62
63 int main(int argc, char* argv[]) {
64     if (argc != 5) {
65         cerr << "Usage: " << argv[0] << " <number_of_disks> <from> <to> <aux>\n"
66         ;
67         return 1;
68     }
69
70     int n = atoi(argv[1]);
71     char from = argv[2][0];
72     char to = argv[3][0];
73     char aux = argv[4][0];
74
75     if (n <= 0) {
76         cerr << "Error: number of disks must be positive.\n";
77         return 1;
78     }

```

```

77
78     solveHanoi(n, from, to, aux);
79     printMoves();
80     clearList();
81
82     return 0;
83 }

```

Листинг 2: Исходный код задачи Ханойской башни

Сборка и запуск

```

1 $ g++ src/hanoi.cpp -o build/hanoi
2
3 ./build/hanoi 4 A C B
4 1. Move disk 1 from A to B
5 2. Move disk 2 from A to C
6 3. Move disk 1 from B to C
7 4. Move disk 3 from A to B
8 5. Move disk 1 from C to A
9 6. Move disk 2 from C to B
10 7. Move disk 1 from A to B
11 8. Move disk 4 from A to C
12 9. Move disk 1 from B to C
13 10. Move disk 2 from B to A
14 11. Move disk 1 from C to A
15 12. Move disk 3 from B to C
16 13. Move disk 1 from A to B
17 14. Move disk 2 from A to C
18 15. Move disk 1 from B to C
19
20 ./build/hanoi 2 M O S
21 1. Move disk 1 from M to S
22 2. Move disk 2 from M to O
23 3. Move disk 1 from S to O
24
25 ./build/hanoi 1 N M K
26 1. Move disk 1 from N to M

```

Результаты

Программа корректно реализует классическое рекурсивное решение задачи Ханойской башни. Каждое движение фиксируется в двусвязный список, список выводится по завершении алгоритма. Память освобождается корректно.

Общее число шагов при n кольцах соответствует формуле $2^n - 1$, что является оптимальным количеством перемещений.